



Computer Games Development CW208
GDD
Year IV

Emily Breen
C00288668
11/04/2026

Game Overview

Can I Hinder? is a crowd-interactive 2.5D top-down action-adventure game developed in C++ using SFML 3.0, supported by a backend API and a Progressive Web App (PWA) that enables real-time audience participation.

In the core gameplay experience, a single player navigates through a series of interconnected maps, engaging in combat with enemies, collecting items that culminate in a climatic boss battle. The game follows a structured progression system, combining exploration, combat and resource management to maintain player interest.

What makes Can I hinder stand out from a traditional action adventure game is the integration of a live spectator interaction layer. External participants can connect to the game via a PWA interface allowing them to influence the game in real time. Spectators are presented with choices that enable them to either assist the player (buffs) or hinder the player(spawning enemies, de-buffs, spawning obstacles)

This introduces an additional layer of unpredictability and emergent gameplay, transforming a single player experience to a hybrid interactive system that blends gameplay with live audience participation. It makes every experience unique.

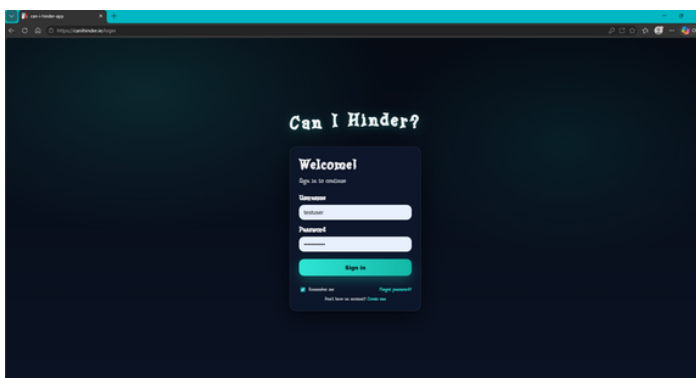


Figure 1 - PWA start screen

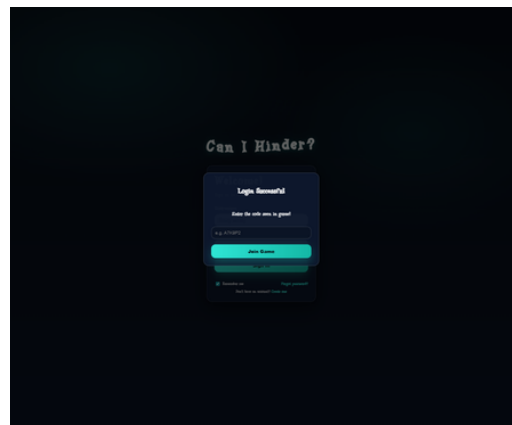


Figure 1.1 - sessionID entry

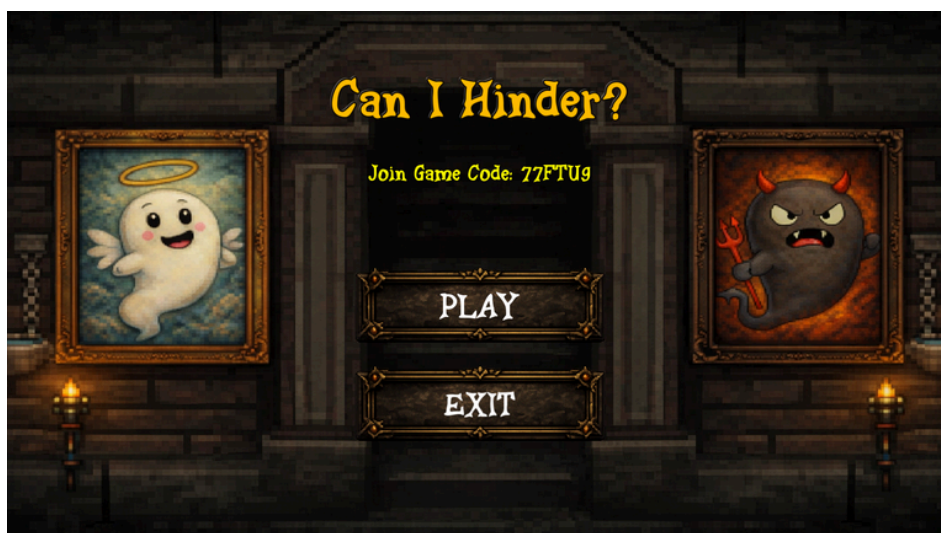


Figure 1.2 -C ++ SFML start screen

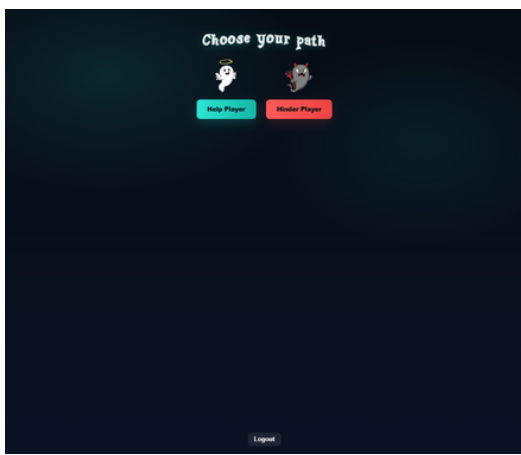


Figure 2 -PWA in game help/hinder buttons



Figure 2.1 -PWA in game help buttons

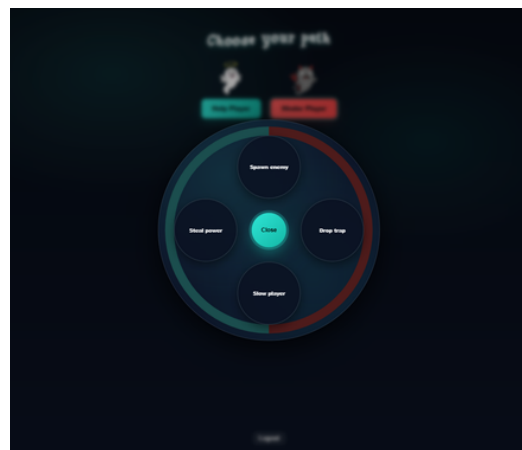


Figure 2.2 -PWA in game hinder buttons

Feature Set

Core Game Features

- Session based crowd interaction: Spectators apply buffs and debuffs via WebSocket commands
- Session code displayed on the main menu and in game HUD for spectators to join at the start or during gameplay.
- Tiled based maps created in Tiled Map Editor, loaded at runtime via TMXlite.
- Four distinct gameplay maps, each with unique item populations.
- Full game state machine: Main menu - Gameplay - Boss Dialogue - Boss Battle - Game Over / Game Won
- Pause menu accessible at any point during gameplay.
- HUD displaying health bar, session code, hotbar and chat box detailing which Spectator issued what buff/debuff.
- Xbox controller support: auto detected on startup, used in preference to keyboard if connected.
- Custom audio system with per state background music and SFX



Figure 3 -SFML in game debuffs being applied



Figure 3.1 -SFML in game debuffs being applied
Figure 3.1 -SFML in game debuffs being applied



3.2 Unlocked Hinder action spawn Brute



3.2 Unlocked Hinder action God Mode

Help Actions (Spectator)

- heal_player: restores a portion of the players health
- speed_up_player: temporarily increases movement speed
- power_boost: temporarily increase attack damage
- sheild_player: temporarily reduces damage recieved
- god_mode: Unlocked after 5 help actions by a spectator: fully heals the player, applies all buffs and cancels all debuffs for 6 seconds.

HinderActions (Spectator)

- spawn_enemy: spawns either a Skeleton or Goblin enemy near the player
- slow_player: temporarily decreases movement speed
- steal_power: temporarily decreases attack damage
- drop_trap: temporarily places an obstacle near the player, impeding progress.
- spawn_brute: Unlocked after 5 hinder actions by a spectator: spawns a Brute type enemy near the player, significantly stronger than the standard enemies.

Networking & infrastructure

- Node.js Websocket relay server with per session message fanout.
- ASP.Net Core REST API: user registration, JWT login, session code generation, password reset.
- Secure WebSocket (wss://) via OpenSSL for Azure hosted deployment
- React 19/Typescript PWA with radial spectator menus and full auth flow.

Gameplay

Core loop

The player spawns on Map 1 and must fight their way through multiple maps, whilst collecting the required keys to unlock doors, to reach the final boss. Spectators watching the session connect via PWA using the session code. The radial menu lets them pick Help or Hinder actions, each on a per action cooldown. Every spectator action is sent to the game in real time and immediately affects gameplay.

Player Actions

- Movement: 8 directional WASD movement. If a controller (xbox) is connected prior to a game session starting the game will automatically choose this.
- Attack: melee slash with directional hitbox (space for keyboard, A for controller)
- Hotbar select and use collected items (health potions, speed potions, power potions) also it displays the keys collected (three in total per map)
- Pause - pause menu toggle

Win/Loss Conditions

Victory	Defeat the final boss in the boss battle encounter
Defeat	Player health reaches zero at any point in the gameplay

Spectator Message Format

All spectator actions are sent as JSON over WebSocket:

```
Sent: ► {user: 'testuser', action: 'hinder', effect: 'spawn_enemy'}  
Message from server: {"user":"testuser","action":"hinder","effect":"spawn_enemy"}
```

Figure - Hinder message format

```
Sent: ► {user: 'testuser', action: 'help', effect: 'heal_player'}
Message from server: {"user":"testuser","action":"help","effect":"heal_player"}
```

Figure - Help message format

The Game World

Overview

The world consists of discrete tile based maps loaded from .tmx files. A Camera system keeps the viewport centered on the player as they traverse the maps, which are x3 larger than the screen. Each map has its own layout and key distribution and item population. Item and keySpawn positions are defined in designated Tiled Object layers and loaded at runtime by the MapRenderer.

Key Locations

Main Menu Screen	What the player is first greeted with, with animated title text, session code display and menu buttons
Maps 1-3	Progressive combat arenas, with keys and items. This is where crowd interaction has maximum impact (spawning enemies, healing player) Each map requires 3 keys to unlock the door to the next area
Map 4	Dedicated arena for the boss encounter with boss, dialogue sequence leads boss battle
Final Level / Boss Battle	Turn based battle with the final boss. Player options: Attack, Magic and Item. Items collected from earlier maps can be used here.

Scaling & Rendering

- Virtual resolution: 1920 x1080 - rendered to a scalable SFML RenderWindow.
- Camera follows the player smoothly, view is constrained to map bounds.
- Tile rendering uses a MapRenderer class wrapping TMXLite parsing.
- Collision geometry is derived directly from designated tile layers.
- item spawn positions(keys, health potions, speed potions, power potions) are loaded from Tiled object layers per map.

Game Characters

Player Character

A top down sprite animated hero with directional movement, melee combat and full buff / debuff support. Visual feedback includes colour flashing on damage and tint changes for active effects.



Figure 4 Player character in game

Base Move Speed	300 units per second
Base Damage	0.25 per hit
Attack Duration	0.60 seconds
Invulnerability	Post hit invulnerability window (to stop stun locking)
Death Duration	0.5 seconds (to play animation before Game Over)
Shield Reduction	50% damage reduction when shield is active
Speed Increase	50% speed increase once active
Power Increase	75% damage increase once active
God Mode Duration	6.0 seconds

Enemy Types

Skeleton/Goblin	Standard enemy tier: both use Pursue steering behaviour for aggressive player tracking. spawn_enemy randomly selects between the two. Low health, standard attack range
Brute	Heavy melee enemy: slower movement but significantly higher damage. Unlocked as a spectator action after 5 hinder actions are accumulated by a single spectator
Boss	Final encounter enemy. Two attack during turn based battle: normal melee and magic (Fireball, Thunder, Ice and Gravity chosen randomly each turn). Preceded by a dialogue sequence



Figure 5 Skeleton/Goblin enemies

Figure 5.1 Brute/Boss enemies

AI Behaviour System

Enemy Movement is driven by an autonomous AIBehaviour class implementing five steering behaviours:

- Seek: moves directly toward the target position at maximum speed.
- Arrive: like Seek but decelerates as the agent nears the target
- Pursue: predicts the players future position before seeking it
- Wander: causes enemies to move in a random but natural way, creating unpredictable behaviour when idle.

All enemies except the boss also apply wall avoidance (preventing passage through collision geometry) and NPC separation (preventing enemy bunching). Standard enemies use pursue when in detection range and wander when idle

User Interface

In game HUD

- Player health bar: normalised 0.0-1.0, utilizing colour transitions with damage
- Session code display: always visable for spectators to join
- Hotbar: has 6 slots 0-2 are for displaying the keys, slot 3 is for healthPotions, slot 4 is for speedPotions and slot 5 is for powerPotions.
- Chat box overlay: scrolling feed of a max of 10 messages with a 6 second time to live, this shows which spectator sent which buff/debuff in real time. Help is displayed in green and hinder in red.

Main Menu	Play, Exit buttons, animated title and session code shown
Pause menu	Semi-transparent overlay
Game Over	Defeat background: Retry / Exit options: displays final stats
Boss Dialogue	Scripted text sequence before the boss battle begins



Figure 6 Paused Menu



Figure 6.1 Game Over Screen

Turn Based Boss Battle

The final encounter departs from the overworld real time gameplay. When the boss fight is triggered (following a dialogue sequence in Map 4), the game transitions into a dedicated turn based battle managed by the FinalLevel class. The class has its own Hp/Mp pools, turn state machine and UI layer, fully separate from the over world

Turn State Machine

PLAYER_CHOICE	Player is presented with three options: Attack, Magic, Item. Navigated with directional input and confirmed with Menu Select (Enter)
PLAYER_ACTION	Selected action is executed. Attack triggers the players melee animation. Magic opens a sub-menu of four spells, each consuming MP. Item option is available and carries over from the earlier maps
ENEMY_ACTION	Boss takes its turn and randomly chooses between a physical attack or magic spell
WIN	Boss HP reaches zero and victory sequence and fade to game over(Win)
LOSE	Player HP reaches zero and death animation and fade to game over(lose)

Player Spells & MP Costs

Fireball	10 MP - animated projectile travelling toward boss
Thunder	10 MP - tall lightning strike descends on the boss
Ice	8 MP - Ice burst effect on the boss
Heal	8 MP - restores player HP, no damage is dealt

If the player has insufficient MP a “Not enough MP!” banner is shown and the action is cancelled, this doesnt consume a turn.

Battle Stats

Player HP	100 (integer this is seperate from the overworld normalised float health)
Player MP	78
Boss HP	100
Action Duration	0.6 seconds per action phase before damage resolves
Damage Text	Floating damage numbers spawn on each hit
Spell Banner	Named spell banner displayed briefly on screen whenever a spell or Game Over or Victory happens



Figure 7 Boss Dialogue Sequence



Figure 7.1 Boss Battle Player using Magic



Figure 7.2 Boss Battle Boss using Magic

Effects System

All visual effects in the game, both spectator triggered gameplay effects and in battle spell animations are managed by the Effects Class. Each effect has a texture, animated sprite, lifetime , optional velocity and optional player-follow behaviour.

Effect Types

Spawn Enemy	Poof animation at spawn point. NPC spawns when animation completes
Fireball	Projectile - travels from caster toward target over 2.0 seconds
FireHit	Impact flash at hit position for 1.0 second
Thunder	Tall lightning bolt striking from above target for 1.5 seconds
ThunderHit	Hit flash at strike point for 1 second
Ice	Ice burst effect for 1 second
IceHit	Ice impact effect for 1 second
Gravity	Projectile gravity ball effect for 2 seconds
GravityHit	Gravity impact for 1 second
Healing	Tall heal wind that follows player for 1 second
SpeedBoost	Quick burst that follows player for 0.6 seconds
PowerBoost	Quick burst that follows player for 0.6 seconds
StealPower	Quick burst that follows player for 0.6 seconds
SlowPlayer	Quick burst that follows player for 0.6 seconds
ShieldPlayer	Shield aura that follows player for the 6 seconds
God Mode	God Mode activation burst for 0.5 seconds
SpawnTrap	Poof effect renders at trap placement position

Musical Scores & Sound Effects

Background Music Tracks

Main Menu	Main Menu.ogg
Gameplay Maps 1-3	Main game music.ogg
Map 4	Map 4 music.ogg
Boss Encounter	Final Boss.ogg
Game Over (defeat)	Game Over music.ogg
Game Over (victory)	Game Over Win music.ogg
Credits	Credits.ogg

Sound Effects

Player Attack	sword slash.ogg
Player Movement	walking effect.ogg
Skeleton movement	Skeleton.ogg
Heal	Heal.ogg
Fireball	Fireball.ogg
Gravity	Gravity.ogg
Ice	Ice.ogg
Thunder	Thunder.ogg

MagicImpact	MagicImpact.ogg
MagicImpact2	MagicImpact2.ogg
MagicImpact3	MagicImpact3.ogg

Crowd Interaction & Multiplayer

Architectual Overview

Can I hinder? is fundamentally a single player game augmented by a real time spectator layer. The crowd interaction system uses a four component architecture:

- C ++ game client: connects to the relay as a WebSocket client
- Node.js relay server: groups clients by session ID and broadcasts within each session.
- ASP.NET Core REST API: Handles Authentication and session code generation.
- React/Typescript PWA: spectator facing interface with radial action menus.

Session Flow

- Game launches: C++ client requests a session code from the REST API.
- Session code is displayed prominently on the main menu and in game.
- Spectator opens the PWA, logs in and enters the session code
- PWA connects to the WebSocket relay at ws[s]://host?session=<CODE>
- Spectator selects an action from the radial Help or Hinder menu
- JSON message is sent to the relay, which rebroadcasts to the game client in the same session
- Game client parses the effect field and applies the gameplay change immediately
- A counter tracks how many helps and hinders each spectator has sent if the threshold of 5 is reached, a new action is unlocked in the PWA radial menu (god_mode for helps; spawn_brute for hinders)
- Spectator's chosen action enters a client side cooldown before it can be used again

Bidirectional Communication

The WebSocket connection enables bidirectional communication between the game and spectators, allowing player actions to influence gameplay in real time. As spectators interact with the game, their actions contribute toward unlocking more powerful abilities such as spawn_brute and god_mode. Once used, these abilities reset the progression system, encouraging continued participation and maintaining engagement

```
Message from server: {"user":"testuser","action":"hinder","effect":"drop_trap"} index-DTmrTGpl.js:60
Message from server: {"hinderCount":5,"type":"progress","unlock":"spawn_brute","user":"testuser"} index-DTmrTGpl.js:60
Hinder count: 5 index-DTmrTGpl.js:60
Unlocked: spawn_brute index-DTmrTGpl.js:60
```

```
Message from server: {"type":"reset","action":"hinder"} pwa-client.tsx:40
```

Session Parameters

Max Spectators	No hard limit, the relay broadcasts to all in session clients estimated 200-500 can join before another server is needed
Action cooldown	Client side per action cooldown enforced in the PWA (rate-limit.tsx)
Authentication	JWT based login required before accessing spectator controls
Connection Mode	ws:// for local testing, wss:// (OpenSSL/Azure) for hosted deployment switch via USE_LOCAL_WS compile flag in game